

CowControl App

Explicación de cada función — Primera versión (v1)

Documento de especificación funcional y técnica, para revisar antes de escribir el código de la aplicación en Flutter.

Versión 2026-07-13

Índice

1. Alcance de esta primera versión
2. Arquitectura general del sistema
3. Protocolos y comunicación entre componentes
4. Estructura de carpetas del proyecto Flutter
5. Pantallas de la aplicación (0 a 5)
6. Diccionario de campos por tabla
7. Conteo en tiempo real — cómo funciona
8. Multi-tenant — cada productor con sus propios datos
9. Paleta visual
10. Qué queda afuera de esta versión

1. Alcance de esta primera versión

La carpeta técnica original de CowControl (PDF con la fundamentación, objetivos y mockups) muestra pantallas con bastantes más datos por animal de los que finalmente se van a mostrar acá. Ese mockup se usó como punto de partida, pero se llevó una versión recortada a productores del **INTA** (Instituto Nacional de Tecnología Agropecuaria) para validarlo, y confirmaron que la mayoría de esos campos no hacen falta.

La única categoría nueva que pidieron agregar fue **condición corporal** (reemplaza al campo "Estado" del mockup original). El resto de los campos del mockup — nombre, sexo explícito, maternidad, preñez, fertilidad, crías del año, alertas puntuales — **no se muestran como campos estructurados** en esta versión. Si un productor necesita anotar algo de eso, lo hace a mano en el campo de observaciones.

Tampoco se muestra la **edad del animal**: los mismos productores señalaron que la edad no se calcula de forma confiable a partir de una fecha de nacimiento cargada a mano — en la práctica se estima mirando la dentadura del animal. Como no aporta al objetivo central (el conteo), se decidió no incluirla y eliminar la columna `fecha_nacimiento` de la base.

Función principal de la app: el conteo. El productor ve la planilla de su ganado, presiona "Iniciar conteo", y a medida que cada animal pasa por el lector RFID/NFC montado en el brete, la app lo marca automáticamente en tiempo real. El resto de las pantallas (detalle de animal, historial) son de apoyo a esa función central.

2. Arquitectura general del sistema

```
Tag NFC/RFID (caravana)
  | el animal pasa por el lector
  v
Lector montado en el brete (RFID 134.2 kHz)
  | la Raspberry Pi procesa la lectura (Middleware)
  v
Raspberry Pi --POST (service_role key)--> Supabase (base de datos en la nube)
                                         |
                                         | Realtime (websocket) + consultas
normales                               v
                                         App CowControl (Flutter)
                                         usa la anon key + login del productor
```

La app **nunca se conecta directo con la Raspberry Pi** — todo pasa por Supabase. La Raspberry Pi usa una clave especial (`service_role`) que le permite escribir lecturas sin restricciones. La app usa

una clave pública (`anon`) combinada con el login del productor, y solo puede ver/editar los datos que le pertenecen a ese productor (ver sección 8).

3. Protocolos y comunicación entre componentes

El sistema tiene varios "saltos" entre un componente y el siguiente, y cada uno usa una tecnología de comunicación distinta. Esta sección existe para que todo el equipo — no solo quien programa la app — entienda exactamente qué pasa en cada tramo, qué formato de datos viaja y quién tiene permiso de hacer qué.

Tramo	Protocolo / tecnología	Qué pasa ahí
Caravana (tag) ↔ Lector	Radiofrecuencia 134.2 kHz (LF), estándar ISO 11784/11785 — FDX-B	El tag es pasivo (no tiene batería): el lector genera un campo electromagnético que "despierta" el chip de la caravana, y el chip responde modulando su propia señal con el número de identificación (CUIG). Es el mismo estándar que usan las caravanas electrónicas oficiales en Argentina.
Lector ↔ Raspberry Pi	Conexión física serial (según el módulo: SPI, UART o USB)	El lector entrega el número de tag ya demodulado a la Raspberry Pi. Ahí corre el Middleware (sobre Linux): descarta lecturas duplicadas del mismo tag en un intervalo corto, para no contar dos veces el mismo pasaje, y arma el paquete de datos a enviar.
Raspberry Pi → Supabase	HTTPS, método POST , cuerpo en JSON	La Raspberry Pi hace una petición POST a <code>/rest/v1/datos_lectura</code> , autenticada con la <code>service_role</code> key en las cabeceras <code>apikey</code> y <code>Authorization: Bearer</code> . Esa clave le da permiso total y no pasa por las políticas de RLS (sección 8) — es el propio sistema el que escribe, no un productor logueado.
App → Supabase Auth (login/registro)	HTTPS, método POST	La app manda el email + contraseña a los endpoints de autenticación de Supabase. Si son correctos, responde con un JWT (JSON Web Token, un token firmado que identifica al productor). La app lo guarda en el dispositivo y lo reenvía en cada pedido siguiente.
App ↔ Supabase (leer/escribir datos)	HTTPS, métodos GET / POST / PATCH , formato JSON	Supabase expone automáticamente una API REST (PostgREST) sobre las 4 tablas. Por ejemplo, la app pide <code>GET /rest/v1/datos_animales?campo_id=eq.<uuid></code> para traer la planilla. Cada pedido incluye el JWT del productor — eso es lo que usa Postgres para aplicar las políticas RLS y devolver solo lo que le pertenece.
App ↔ Supabase Realtime (conteo en vivo)	WebSocket (WSS)	A diferencia de los pedidos anteriores, acá la conexión queda abierta todo el tiempo que dura el conteo. Supabase Realtime escucha los cambios que Postgres registra en su WAL (write-ahead log, el mecanismo interno con el que Postgres anota cada modificación) y los reenvía por ese canal abierto a todos los clientes suscriptos. Así la app se entera al instante de una lectura nueva, sin tener que preguntar "¿hay algo nuevo?" cada tanto (lo que se conoce como <i>polling</i>).

```

Caravana (tag pasivo)
  | radiofrecuencia 134.2 kHz, protocolo FDX-B (ISO 11784/11785)
  v
Lector RFID en el brete
  | conexion serial (SPI/UART/USB) -- la Raspberry Pi decodifica la trama
  v
Raspberry Pi (Middleware, Linux)
  | HTTPS POST, JSON, header "Authorization: Bearer <service_role key>"
  v
Supabase (PostgREST) ---guarda en---> Postgres (tabla datos_lectura)
  |
  | Postgres registra el cambio en su WAL (write-ahead log)
  v
Supabase Realtime ---WebSocket (WSS)---> App CowControl (conteo en vivo)

App CowControl (Flutter)
  | HTTPS POST (login/registro)
  v
Supabase Auth ---responde con un JWT---> App CowControl
  |
  | ese JWT viaja en el header "Authorization: Bearer <JWT>"
  v
App CowControl <---HTTPS GET/POST/PATCH---> Supabase (PostgREST)

```

Cómo se conectan la autenticación y los permisos (JWT + RLS)

Esta parte es clave para entender por qué el sistema es seguro sin que la app tenga que "acordarse" de filtrar nada:

1. Cuando el productor inicia sesión, Supabase Auth le devuelve un **JWT** que contiene, entre otras cosas, su `sub` (el UUID único de su cuenta).
2. La app guarda ese JWT y lo manda en la cabecera `Authorization` de *todos* los pedidos siguientes (tanto REST como la conexión Realtime).
3. Dentro de Postgres, la función `auth.uid()` lee ese UUID directamente del JWT que llegó en el pedido — no hay que pasarlo "a mano" en ningún parámetro.
4. Las políticas RLS (ver sección 8) comparan ese `auth.uid()` contra el `productor_id` de cada fila, y la base de datos misma decide qué filas devolver o dejar escribir. Aunque hubiera un error de programación en la app, la base de datos igual bloquearía el acceso a datos de otro productor — el filtro no depende de que el código de la app esté bien escrito.

La Raspberry Pi queda **fuera** de este esquema a propósito: no inicia sesión como un productor, usa la `service_role` key, que es una clave de sistema que Postgres reconoce como "superusuario" y que ignora las políticas RLS por completo.

4. Estructura de carpetas del proyecto Flutter

Así se va a organizar el código fuente dentro de `app_cowcontrol1/` una vez que arranquemos a programar:

```

app_cowcontrol/
|-- lib/
|   |-- main.dart                -> arranca la app, inicializa Supabase con las
|   |                           credenciales del .env, define las rutas
|   |-- theme/
|   |   |-- app_theme.dart       -> paleta de colores marron/tierra y estilos
|   |   |                           compartidos por toda la app
|   |-- models/
|   |   |-- campo.dart           -> representa una fila de datos_campo
|   |   |-- animal.dart          -> representa una fila de datos_animales
|   |   |-- lectura.dart         -> representa una fila de datos_lectura
|   |   |-- vacuna.dart          -> representa una fila de datos_vacuna
|   |-- services/
|   |   |-- supabase_service.dart -> cliente unico de Supabase, usado por
|   |   |                           todos los demas servicios
|   |   |-- auth_service.dart    -> login, registro, cerrar sesion
|   |   |-- campos_service.dart  -> traer/crear campos del productor logueado
|   |   |-- animales_service.dart -> traer animales de un campo, traer el
|   |   |                           detalle de uno en particular
|   |   |-- lecturas_service.dart -> traer historial de lecturas + suscripcion
|   |   |                           Realtime para el conteo en vivo
|   |   |-- vacunas_service.dart -> traer la ultima vacuna aplicada a un animal
|   |-- screens/
|   |   |-- login_screen.dart     -> Pantalla 0
|   |   |-- selector_campo_screen.dart -> Pantalla 1
|   |   |-- planilla_screen.dart  -> Pantalla 2
|   |   |-- conteo_screen.dart    -> Pantalla 3
|   |   |-- detalle_animal_screen.dart -> Pantalla 4
|   |   |-- historial_screen.dart -> Pantalla 5
|   |-- widgets/
|   |   |-- animal_card.dart      -> tarjeta de un animal (se reusa en
|   |   |                           planilla y en conteo en vivo)
|   |   |-- categoria_badge.dart  -> icono/etiqueta vaca-toro-ternero
|   |   |-- condicion_corporal_chip.dart -> indicador visual de 1 a 5
|-- .env                          -> SUPABASE_URL y SUPABASE_ANON_KEY
|                               (no se sube a git)
|-- .gitignore                   -> incluye .env
|-- pubspec.yaml                 -> dependencias: supabase_flutter,
|                               flutter_dotenv
|-- android/ ios/ web/ ...       -> generadas automaticamente por
|                               "flutter create", no se tocan a mano
|-- memory.md
|-- cosas_pendientes.md
|-- explicacion_cada_funcion.pdf -> este documento

```

Cada archivo de `services/` es el único lugar del código que habla directo con Supabase para su tabla correspondiente — las pantallas nunca arman consultas SQL/HTTP a mano, siempre le piden los datos al servicio correspondiente. Esto es lo que en la presentación se puede explicar como "separar la lógica de datos de la lógica visual".

5. Pantallas de la aplicación

Pantalla 0 — Login / Registro

Qué hace: primera pantalla que ve cualquiera al abrir la app. Formulario de email + contraseña, con la opción de "Ya tengo cuenta" (iniciar sesión) o "Soy nuevo" (registrarme). Usa Supabase Auth.

Qué pasa después de loguearse: si el productor no tiene ningún campo cargado todavía (primera vez que usa la app), se lo lleva a un formulario simple para crear su primer campo (nombre + ubicación). Si ya tiene uno o más, pasa a la Pantalla 1.

Tabla relacionada: ninguna directamente — usa el sistema de autenticación de Supabase (`auth.users`), no una tabla propia del proyecto.

Pantalla 1 — Selector de campo

Qué hace: muestra la lista de campos (establecimientos) que le pertenecen al productor logueado — puede tener uno o varios. Si solo tiene uno, la app lo selecciona automáticamente y pasa directo a la Pantalla 2, sin detenerse acá.

Acciones: tocar un campo para entrar a su planilla. Botón para crear un campo nuevo.

Tabla: `datos_campo`, filtrada automáticamente por Row Level Security para mostrar solo los campos del productor actual.

Pantalla 2 — Planilla general

Qué hace: es la pantalla principal de la app. Lista todos los animales vivos del campo seleccionado. Por cada animal se muestra:

- **ID** el código del tag (`rfid_uid`)
- **Categoría** vaca / toro / ternero / ternera / vaquillona, mostrada como ícono o etiqueta de color (no como texto "Sexo: ...")
- **Condición corporal** escala de 1 a 5

Acciones: buscar/filtrar animales (por categoría o rango de condición corporal), tocar un animal para ver su detalle (Pantalla 4), botón grande y visible **"Iniciar conteo"** que lleva a la Pantalla 3.

Tabla: `datos_animales`, filtrada por `campo_id` y por `estado_vital = 'vivo'`.

Pantalla 3 — Conteo en vivo

Qué hace: versión reducida de la planilla, pensada para usarse mientras el operario hace pasar animales por el brete. Por cada animal se muestra solo **categoría + ID**, y aparece una tilde apenas ese animal es detectado por el lector — sin que el usuario tenga que refrescar nada (ver secciones 3 y 7, Realtime).

Acciones: "Detener" (pausa el conteo sin perder lo ya marcado), "Reanudar", "Finalizar" (termina el conteo). "Ver más" en cualquier fila lleva al detalle completo de ese animal (Pantalla 4), sin interrumpir el conteo en curso.

Tabla: `datos_animales` (para la lista completa) + escucha en tiempo real sobre `datos_lectura` (para saber quién ya pasó).

No hay todavía un botón "Guardar/Borrar el conteo completo" como unidad — cada lectura ya queda guardada individualmente en `datos_lectura` apenas ocurre. Agrupar lecturas en una "sesión de conteo" guardable/descartable queda para una versión futura (ver `cosas_pendientes.md`).

Pantalla 4 — Detalle de animal

Qué hace: ficha completa de un animal en particular, a la que se llega tocando "Ver más" desde la Pantalla 2 o la Pantalla 3. Muestra:

- **ID** `rfid_uid`
- **Condición corporal** escala de 1 a 5
- **Vacunas** resumen simple: la última vacuna aplicada y su fecha (no el historial completo)
- **Observaciones** texto libre que el productor carga a mano — acá va todo lo que no tiene un campo propio (por ejemplo, si está preñada, una lesión puntual, etc.)

Tablas: `datos_animales` (datos propios) + `datos_vacuna` (se pide solo el registro más reciente de ese animal).

Pantalla 5 — Historial de lecturas

Qué hace: lista las últimas lecturas registradas en el campo actual, ordenadas de la más reciente a la más vieja (fecha + qué animal fue detectado).

Tabla: `datos_lectura`, filtrada por `campo_id` y ordenada por `fecha_hora` descendente.

6. Diccionario de campos por tabla

Solo se documentan acá las columnas que la app v1 realmente usa. Las tablas en Supabase pueden tener alguna columna extra (como `nombre` o `raza` en `datos_animales`) que existen en la base pero no se muestran en esta versión.

datos_campo

Campo	Para qué sirve
<code>id</code>	Identificador único del campo (UUID). Lo usan las demás tablas para referenciarlo, nunca se muestra al usuario.
<code>nombre_campo</code>	Nombre del establecimiento, puesto por el productor. Ej: "Villa Langostura".
<code>ubicacion</code>	Provincia o localidad del campo. Ej: "Chaco".
<code>productor_id</code>	<i>(migración pendiente)</i> Referencia a la cuenta del productor dueño de este campo. Es la columna que hace posible que cada productor solo vea lo suyo.

datos_animales

Campo	Para qué sirve
<code>id</code>	Identificador único del animal (UUID). Lo usan <code>datos_lectura</code> y <code>datos_vacuna</code> para referenciarlo.
<code>rfid_uid</code>	Código del tag NFC/RFID físico. Es el "ID" que se muestra en toda la app.
<code>sexo</code>	En esta versión se usa como "categoría del animal": vaca / toro / ternero / ternera / vaquillona. Se muestra como badge/ícono, nunca como texto "Sexo: ...".
<code>estado_vital</code>	"vivo" o "muerto". La planilla y el conteo solo muestran animales vivos por defecto.
<code>condicion_corporal</code>	<i>(migración pendiente)</i> Escala de 1 a 5, con medios puntos (1 = muy flaca, 5 = muy gorda). Reemplaza al campo "Estado" del mockup original.
<code>observaciones</code>	Texto libre del productor. Acá se anota cualquier dato particular que no tiene un campo propio (preñez, una lesión, etc.).
<code>campo_id</code>	A qué campo pertenece este animal. Filtra qué animales se muestran según el campo seleccionado.

datos_lectura

Campo	Para qué sirve
id	Identificador único de la lectura. Se crea uno nuevo cada vez que un tag pasa por el lector.
rfid_uid	Código leído en ese momento. Se guarda siempre, incluso si el animal no está registrado.
id_animal	A qué animal corresponde el tag leído (la Raspberry Pi lo busca por <code>rfid_uid</code>). Puede quedar vacío si el tag no está registrado.
campo_id	En qué campo se hizo la lectura. Define qué conteo se actualiza en tiempo real.
fecha_hora	Momento exacto de la lectura. Es lo que usa la Pantalla 5 para ordenar el historial.

datos_vacuna

Campo	Para qué sirve
id_animal	A qué animal corresponde esta vacuna. La app pide solo el registro con la <code>fecha_vacuna</code> más reciente por animal, para el resumen de la Pantalla 4.
nombre_vacuna	Nombre de la vacuna aplicada. Ej: "Aftosa".
fecha_vacuna	Cuándo se aplicó. Junto con <code>nombre_vacuna</code> , arma el resumen "Última vacuna: Aftosa — 12/03/2026".

7. Conteo en tiempo real — cómo funciona

Esta sección es la aplicación concreta, dentro de la app, del protocolo WebSocket/Realtime explicado en la sección 3.

Cuando el productor toca "Iniciar conteo" (Pantalla 3), la app abre una **suscripción Realtime** de Supabase sobre la tabla `datos_lectura`, filtrada por el `campo_id` actual. Esto es una conexión tipo websocket: en vez de la app preguntando cada tanto "¿hay algo nuevo?" (polling), es Supabase quien le avisa a la app apenas se inserta una fila nueva.

Cuando llega ese aviso, la app busca en la lista de animales en pantalla cuál tiene el mismo `rfid_uid` (o `id_animal`) que la lectura recién insertada, y le pone la tilde. Todo esto pasa sin que el usuario tenga que hacer nada — es lo que en la carpeta técnica se describe como "sincronización" entre el lector y la app.

8. Multi-tenant — cada productor con sus propios datos

La app va a poder ser usada por varios productores distintos, cada uno con su propio ganado. En vez de crear un proyecto de Supabase separado por productor (complicado de mantener), se usa un **único proyecto de Supabase** con dos mecanismos combinados:

- **Supabase Auth:** cada productor tiene su cuenta (email + contraseña).
- **Row Level Security (RLS):** reglas de seguridad a nivel de base de datos que hacen que, aunque todos los productores compartan las mismas 4 tablas, cada uno *solo pueda ver y modificar* las filas que le pertenecen (a través de la columna `productor_id` en `datos_campo`, heredada por los animales/lecturas/vacunas de ese campo).

La Raspberry Pi no se ve afectada por esto: sigue insertando lecturas con una clave especial (`service_role`) que ignora las reglas de RLS, porque no es un "productor logueado" sino el propio sistema.

9. Paleta visual

Colores tierra/marrón, siguiendo la estética del mockup de la carpeta técnica original — el mismo espíritu de este documento (fondo crema, acentos marrón/naranja tostado, texto marrón oscuro). Esto se traduce en el `app_theme.dart` del proyecto Flutter.

10. Qué queda afuera de esta versión

Documentado en detalle en `cosas_pendientes.md`:

- **Sesiones de conteo agrupadas** (guardar/borrar un conteo completo como unidad, ver conteos anteriores como planillas históricas).
- **Cámara vinculada a cada lectura** (ver el video en el momento exacto en que pasó un animal) — el hardware físico todavía no está armado.
- **Resto de campos del mockup original** (maternidad, preñez, fertilidad, crías del año, alertas puntuales) — se evalúan de a uno, solo si un productor real los pide.